

DM548: Computer Architecture
& System Programming
DM588: Computer Architecture

Fall 2026

Makefiles

Introduction

Are you tired of typing build commands all the time?

Java:

1. For each `.java` file, create a `.class` file with `javac`.
2. Create a `jar` file from the `.class` files with `jar`.

Assembly:

1. For each `.s` file, create a `.o` file with `as`.
2. Link the `.o` files to create an executable.

C:

1. For each `.c` file, create a `.o` file with `gcc`.
2. Link the `.o` files to create an executable.

L^AT_EX:

1. For the master `.tex` file, create a `.pdf` file with `pdflatex`.
2. Run `bibtex` on the `.aux` file.
3. Run `pdflatex` a few more times.

Introduction

- ▶ We could create a shell script to automate things.
- ▶ But sometimes we only want to do some of the tasks.
- ▶ And we don't actually need to redo everything all the time.
- ▶ And when we do anything, we would like to parallelise execution.
- ▶ So, only execute tasks “if needed”.
- ▶ If a task fails, propagate failure.
- ▶ Specify dependencies and individual tasks, let a program sort out the details.

A solution:

[S. I. Feldman, (1979), “Make — a program for maintaining computer programs”.

Software: Practice and Experience.]

(Feldman worked at AT&T Bell Labs)

We will use GNU Make, available as the `make` command.

Basics

- ▶ We specify our tasks in a `makefile`, which usually is named “Makefile” and placed in the root of your project.
- ▶ Invoking `make` will search for a makefile in the current folder (trying a few different names).
- ▶ (A specific file can be specified with `-f` (but you shouldn't need that in this course))
- ▶ A makefile defines a set of `targets`.
- ▶ A target is often the name of a file to be created/updated, but can be an invented name (a so-called `phony target`).
- ▶ When invoking `make` we can specify a list of targets to update, otherwise it updates the default target.
- ▶ The default target can be specified in the makefile, but is otherwise the first one appearing.
- ▶ Common phony targets: `all` (build all), `clean` (remove generated files), `doc` (compile documentation).

Makefile Basics

A makefile consists of

- ▶ Explicit rules (rules you write)
- ▶ Implicit rules (rules you don't write)
- ▶ Variable definitions (that may be used by either rule type)
- ▶ Directives
(e.g., makefile inclusion and conditional code selection)
- ▶ Comments (line comments starting with #)

Notes:

- ▶ A makefile is not an imperative script, it declares a rule database.
- ▶ The order of rules doesn't matter.
- ▶ Except, if no default target is declared, the first target is the default.

Explicit Rules

```
targets: prerequisites
    recipe
```

- ▶ “targets”: the targets that gets updated by this rule.
- ▶ “prerequisites”: the targets that must be up to date before the recipe is run.
- ▶ The recipe may span multiple lines.
- ▶ Each line must be indented by a tab character.
- ▶ Each line is executed as shell code.

Example

```
utils.o: utils.s
    as -g -o utils.o utils.s
```

If we modify `utils.s` (or `utils.o` doesn't exist) then the recipe will be run.

<https://www.gnu.org/software/make/manual/make.html#Rules>

Phony Rules

Sometimes it's useful to have a symbolic targets.
Such targets should be declared **phony**:

```
.PHONY: all doc clean  
  
all: myProgram doc  
  
doc: report.pdf  
  
clean:  
    rm -f *.o *.aux
```

(Rules for myProgram program and for report.pdf omitted)

It will probably still work without the phony-declaration,
until someone happens to create a file with that same name.

Makefile for Sorter (take 1 of n)

```
sorter.o: allocate.o fileHandling.o parsing.o print.o sorter.o
    ld -o sorter allocate.o fileHandling.o \
        parsing.o print.o sorter.o

allocate.o: allocate.s
    as -g -o allocate.o allocate.s

fileHandling.o: fileHandling.s
    as -g -o fileHandling.o fileHandling.s

parsing.o: parsing.s
    as -g -o parsing.o parsing.s

print.o: print.s
    as -g -o print.o print.s

sorter.o: sorter.s
    as -g -o sorter.o sorter.s

.PHONY: clean # 'clean' is not a real file
clean:        # so it should be declared phony
    rm -f *.o sorter
```

(Lines can be split by escaping the line break with `\`)

Variables

- ▶ Variables simply contain strings.
- ▶ They can be referenced as either `$(var)` or `${var}`.
- ▶ (If you need to reference a shell variable, escape the `$` with `$`:
`$$${shellVar}`)
- ▶ Two flavours:
 - ▶ Recursively expanded variables.
 - ▶ Simply expanded variables.
- ▶ Assignment determines flavour:
 - ▶ Recursively:
`OBJS = $(givenFiles)$(myFiles)`
 - ▶ Simply:
`OBJS := $(givenFiles)$(myFiles)`
 - ▶ (There are other assignment operators)
- ▶ Unassigned variables are empty (except for a special subset).
- ▶ There are **automatic variables** inside rules (more later).
- ▶ Variables can be set on invocation:
`make flags='-g -O3'`

<https://www.gnu.org/software/make/manual/make.html#Using-Variables>

Variable Flavours

Recursively Expanded:

```
# all variables are empty at this point
foo = $(bar)
# the value of foo is '$(bar)'
bar = $(ugh) # indirectly modifies foo
ugh = Huh? # indirectly modifies bar and therefore foo

all:
    echo $(foo)
```

Simply Expanded:

```
x := foo
y := $(x) bar
# the value of y is not '$(x) bar', but 'foo bar'
x := later # doesn't change y
```

That means, don't do:

```
x = $(x) foo # infinite recursion on expansion
```

Makefile for Sorter (take 2 of n)

```
OBJS = allocate.o fileHandling.o parsing.o print.o sorter.o
flags = -g

sorter: $(OBJS)
    ld -o sorter $(OBJS)

allocate.o: allocate.s
    as $(flags) -o allocate.o allocate.s

fileHandling.o: fileHandling.s
    as $(flags) -o fileHandling.o fileHandling.s

parsing.o: parsing.s
    as $(flags) -o parsing.o parsing.s

print.o: print.s
    as $(flags) -o print.o print.s

sorter.o: sorter.s
    as $(flags) -o sorter.o sorter.s

.PHONY: clean # 'clean' is not a real file
clean:        # so it should be declared phony
    rm -f *.o sorter
```

Automatic Variables

In rules, some variables are automatically set:

- ▶ `$@`: the name of the target of the rule.
- ▶ `$<`: the name of the first prerequisite.
- ▶ `$^`: the space-separated list of all prerequisites.
- ▶ (there are more)

Makefile for Sorter (take 3 of n)

```
OBJS = allocate.o fileHandling.o parsing.o print.o sorter.o
flags = -g

sorter: $(OBJS)
    ld -o sorter $^

allocate.o: allocate.s
    as $(flags) -o $@ $^

fileHandling.o: fileHandling.s
    as $(flags) -o $@ $^

parsing.o: parsing.s
    as $(flags) -o $@ $^

print.o: print.s
    as $(flags) -o $@ $^

sorter.o: sorter.s
    as $(flags) -o $@ $^

.PHONY: clean # 'clean' is not a real file
clean:        # so it should be declared phony
    rm -f *.o sorter
```

Variables Used In Implicit Rules

Some variables have default values and are used in implicit rules:

- ▶ **AS**: the assembler to run (“as”).
- ▶ **ASFLAGS**: extra flags to use when assembling.
- ▶ **CC**: the C compiler to use (“cc”, usually a link to gcc)
- ▶ **CPPFLAGS**: preprocessor flags to use when compiling C/C++.
The -I flags should go here.
- ▶ **CFLAGS**: extra flags to use when compiling C code.
The compiler flags like -g -O2 should go here.
- ▶ ...

Makefile for Sorter (take 4 of n)

```
OBJS = allocate.o fileHandling.o parsing.o print.o sorter.o
ASFLAGS = -g

sorter: $(OBJS)
    ld -o sorter $^

allocate.o: allocate.s
    $(AS) $(ASFLAGS) -o $@ $^

fileHandling.o: fileHandling.s
    $(AS) $(ASFLAGS) -o $@ $^

parsing.o: parsing.s
    $(AS) $(ASFLAGS) -o $@ $^

print.o: print.s
    $(AS) $(ASFLAGS) -o $@ $^

sorter.o: sorter.s
    $(AS) $(ASFLAGS) -o $@ $^

.PHONY: clean # 'clean' is not a real file
clean:        # so it should be declared phony
    rm -f *.o sorter
```

Implicit Rules

If a file needs updating, then `make` will fill in the rule set with built-in **implicit rules** when they don't override an explicit rule.

Example rules:

- ▶ A `.o` file can be created from a `.c` file.
- ▶ A `.o` file can be created from a `.cpp` file.
- ▶ A `.o` file can be created from a `.s` file.

Makefile for Sorter (take 5 of n , the final one)

```
OBJS = allocate.o fileHandling.o parsing.o print.o sorter.o
ASFLAGS = -g

sorter: $(OBJS)
    ld -o sorter $^

.PHONY: clean # 'clean' is not a real file
clean:        # so it should be declared phony
    rm -f *.o sorter
```

Writing Recipes

- ▶ Each line in a recipe is executed as a separate shell command.
- ▶ Escape the line ending to make a multi-line recipe.
- ▶ Recipe lines are executed sequentially in the shell.
- ▶ The variable `SHELL` determines which shell.
- ▶ The exit code of the shell code matters, non-zero means it failed.
- ▶ By default, if some line fails the whole recipe fails.
Put `-` in front of a line to ignore failure.
- ▶ By default the recipes are echoed when executed.
Put `@` in front of a line to suppress echoing.

<https://www.gnu.org/software/make/manual/make.html#Recipes>

Example for C

```
SAN_FLAGS = -fsanitize=address,undefined,leak
CFLAGS = -Wall -g -O2 $(SAN_FLAGS)
LDLIBS = -lm -lstuff
CPPFLAGS = -Ipath/to/stuff/include
LDFLAGS = -Lpath/to/stuff/lib $(SAN_FLAGS)

OBJS = file1.o file2.o file3.o

prog: $(OBJS)
    $(CC) $(LDFLAGS) -o $@ $^ $(LDLIBS)

.PHONY: clean
clean:
    rm -f *.o prog
```